

Healthcare SQL Analysis — Findings Report

25 Queries | 5 Sections | PostgreSQL | 05_analysis_queries.sql

About This Report

This report presents the findings from 25 SQL analysis queries run against a cleaned and normalised PostgreSQL database of 50,000 hospital admissions. Queries are organised across five sections: Aggregations, JOINS, Subqueries & CTEs, Window Functions, and Combined Analysis. Each entry includes the business question, the SQL approach used, and the key finding.

Note: This is a synthetic Kaggle dataset used for portfolio purposes. The unusually even distribution of results across most dimensions is characteristic of synthetically generated data. In a real-world dataset, more pronounced variation would typically be observed.

Executive Summary — Key Findings

#	Finding	Query
1	All 6 conditions each account for ~16–17% of admissions	1.1
2	Average billing is consistent across conditions (\$25k–\$26k)	1.2
3	All 5 insurers handle near-equal claim volumes and revenue	1.3
4	Admission type has minimal impact on length of stay	1.4
5	Test result outcomes are evenly split (~33% each) per condition	1.6
6	Billing gender gap is negligible - under \$500 per condition	1.8
7	~24% of patients have more than one admission	2.4
8	No single insurer dominates any medical condition	3.3
9	Patients aged 45–74 drive the majority of admissions and billing	3.5
10	Insurer revenue rankings are stable year over year	4.1

Section 1 — Aggregations & GROUP BY (Queries 1.1–1.8)

Query 1.1 — Total admissions per medical condition

Question: Which conditions drive the most hospital admissions?

Groups all admissions by condition and calculates each condition's share of total admissions using a window function over the full result set.

Finding: All 6 conditions account for approximately 16–17% each. The dataset is synthetically balanced, but in real-world data this query would immediately surface which conditions drive hospital demand.

Query 1.2 — Average billing amount per medical condition

Question: Which conditions are the most expensive to treat?

Joins admissions to insurance to calculate average, minimum and maximum billing per condition.

Finding: Average billing sits in the \$25,000–\$26,000 range across all conditions with minimal variation, suggesting billing is driven more by length of stay and admission type than by condition alone.

Query 1.3 — Admissions and revenue by insurance provider

Question: Which insurer handles the most claims and generates the most revenue?

Aggregates total claims, total revenue, and average claim value per insurer.

Finding: All 5 insurers handle approximately 20% of claims each with total revenues in a narrow band. No single insurer dominates — average claim values are consistent across all providers.

Query 1.4 — Average length of stay per admission type

Question: Do emergency admissions result in longer stays than elective ones?

Calculates average, minimum, and maximum length of stay in days for Emergency, Urgent, and Elective admissions.

Finding: All three admission types produce similar average stay lengths. In real healthcare data, emergency admissions typically result in longer stays — the flat distribution here is characteristic of synthetic data.

Query 1.5 — Admissions by year and month

Question: Are there seasonal trends in hospital admissions?

Uses **EXTRACT** to pull year and month from admission dates and counts admissions per period.

Finding: Admissions are distributed evenly across months and years with no seasonal peaks. In real hospital data, winter months typically see elevated respiratory and cardiac admissions — this query provides the framework to detect those patterns.

Query 1.6 — Test result distribution per medical condition

Question: Which conditions most frequently produce abnormal test results?

Uses **PARTITION BY** in a window function to calculate each test result as a percentage within its condition group.

Finding: Normal, Abnormal, and Inconclusive results each represent approximately 33% of outcomes per condition. In real clinical data, conditions like cancer would show elevated abnormal rates — this query is the right tool to detect that.

Query 1.7 — Blood type distribution across patients

Question: What is the blood type makeup of the patient population?

Counts distinct patients per blood type with population percentage using a window function.

Finding: All 8 blood types are represented with broadly equal distribution. In real populations O+ is most common (~38%) and AB- is rarest (~1%) — the even split confirms the synthetic nature of the dataset.

Query 1.8 — Average billing by gender and medical condition

Question: Is there a billing difference between genders for the same condition?

Joins all three tables and groups by gender and condition to surface any billing disparity.

Finding: Average billing for the same condition differs by less than \$500 between male and female patients across all six conditions — no meaningful gender billing gap is present.

Section 2 — JOINS Across Multiple Tables (Queries 2.1–2.5)

Query 2.1 — Full patient admission detail

Question: Produce a complete view of each admission joining all three tables.

Joins patients, admissions, and insurance in a single query to produce a master admission view. Calculates length_of_stay_days inline using date arithmetic.

Finding: The three-table JOIN produces a clean, complete record per admission — exactly the kind of view a health insurer case manager would work from daily.

Query 2.2 — High-cost admissions with patient detail

Question: Who are the patients billed above \$50,000 and what conditions do they have?

Filters the three-table JOIN to surface only the highest-cost admissions.

Finding: A small number of admissions exceed \$50,000. All six conditions appear in the high-cost cohort, reinforcing that billing in this dataset is not strongly condition-dependent. In real data this query would identify outlier cases requiring billing review.

Query 2.3 — Abnormal test results by insurer

Question: Which insurance providers are covering the most abnormal-result admissions?

Filters admissions with abnormal test results and groups by insurer and condition to show which providers are most exposed to high-complexity cases.

Finding: Abnormal result admissions are distributed evenly across insurers and conditions. In real insurance data this analysis would be central to risk pool assessment and premium pricing.

Query 2.4 — Patients admitted more than once

Question: Which patients are repeat admissions and what conditions are they returning for?

Uses a subquery with `HAVING COUNT(*) > 1` to identify repeat patients, then surfaces their full admission history.

Finding: Approximately 9,765 patients (~24% of the patient population) have more than one admission, consistent with a chronic conditions dataset. Repeat admissions for the same condition indicate ongoing management — a key metric for insurers assessing long-term cost exposure.

Query 2.5 — Medication usage by medical condition

Question: What medications are most commonly prescribed per condition?

Groups admissions by condition and medication to surface prescribing patterns.

Finding: Multiple medications appear across all conditions with broadly even prescribing rates. In real clinical data this analysis would flag unusual prescribing patterns and check whether evidence-based treatments are being applied consistently.

Section 3 — Subqueries & CTEs (Queries 3.1–3.5)

Query 3.1 — Patients billed above average for their condition

Question: Which patients were charged more than the average for their specific medical condition?

A subquery calculates average billing per condition and is joined as a derived table. The outer query filters and calculates the `above_avg_by` column. This approach runs the subquery once rather than once per row — more efficient than a correlated subquery.

Finding: Roughly half of patients are billed above their condition average by definition. The `above_avg_by` column quantifies the gap — directly transferable to real insurance billing audits.

Query 3.2 — Monthly admissions trend (chained CTE + LAG)

Question: How have admissions trended month over month?

Two chained CTEs: the first aggregates admissions by month using `DATE_TRUNC`, the second applies `LAG()` to retrieve the previous month's count. `COALESCE` handles the `NULL` for the first month. `TO_CHAR` formats the output as 'Mon YYYY'.

Finding: Monthly admissions are consistent across the dataset timeline. The `month_on_month_change` column is the raw material for trend analysis — in real data this would surface surges tied to seasonal illness or capacity events.

Query 3.3 — Top earning insurer per medical condition (CTE + RANK)

Question: Which insurer dominates billing for each condition?

A CTE aggregates revenue by insurer and condition. A second CTE applies `RANK()` partitioned by condition. The outer query filters to `revenue_rank = 1` to return only the top earner per condition.

Finding: Different insurers lead on revenue for different conditions but margins are narrow. No single insurer dominates all conditions. This CTE + RANK pattern is one of the most commonly used in real-world insurance reporting.

Query 3.4 — Patients with longest stays vs condition average (CTE)

Question: Which patients stayed significantly longer than average for their condition?

A CTE calculates average stay per condition. A second CTE joins this back to individual admissions. The outer query calculates `days_above_avg` and filters to patients who exceeded their condition's average.

Finding: Patients with significantly above-average stays represent the highest-cost and most complex cases. In real clinical settings this query would feed directly into case management escalation workflows.

Query 3.5 — Admission risk profile by age band (CTE + CASE)

Question: How do admission patterns and billing vary across patient age groups?

A CTE uses a CASE BETWEEN statement to assign each patient to one of six age bands, then aggregates admissions, unique patients, and billing per band.

Finding: Patients aged 45–74 account for the largest share of admissions and total billing, consistent with higher chronic condition prevalence in older populations. The 75+ band shows higher average billing per admission, reflecting the complexity of care in elderly patients.

Section 4 — Window Functions (Queries 4.1–4.5)

Query 4.1 — Rank insurers by total revenue per year (RANK)

Question: How does each insurer's revenue ranking change from year to year?

EXTRACT pulls the year from the admission date. GROUP BY collapses to one row per insurer per year. RANK() with PARTITION BY year resets the ranking counter for each year, ordering by SUM(billing_amount) DESC.

Finding: Insurer revenue rankings are broadly stable year over year — the same providers appear near the top consistently. This indicates a mature, stable market without significant disruption across the dataset period.

Query 4.2 — Most recent admission per patient (ROW_NUMBER)

Question: What was each patient's most recent admission and how much did it cost?

A CTE applies ROW_NUMBER() partitioned by patient_id and ordered by date descending, assigning rn=1 to the most recent admission. The outer query filters WHERE rn = 1 to return exactly one row per patient.

Finding: ROW_NUMBER() is one of the most practically useful window functions in healthcare analytics — returning the most recent record per entity is a pattern used constantly in insurance claims processing and patient dashboards.

Query 4.3 — Billing percentile rank per condition (PERCENT_RANK + NTILE)

Question: Where does each admission's billing fall within the distribution for its condition?

PERCENT_RANK() and NTILE(4) are both partitioned by condition. PERCENT_RANK() returns double precision in PostgreSQL — a CAST to NUMERIC is required before passing to ROUND(). This is a documented bug fix included as a portfolio talking point.

Finding: Billing is broadly uniformly distributed within each condition. PERCENT_RANK() and NTILE are the standard tools for billing distribution analysis in insurance and healthcare finance.

Query 4.4 — Condition ranking by avg billing within admission type (CTE + RANK)

Question: Within each admission type, which condition is the most expensive?

A CTE pre-calculates average billing per admission type and condition. RANK() is then applied in the outer query. PostgreSQL does not permit window functions nested inside another window function's ORDER BY — the CTE is the correct solution to this constraint.

Finding: Condition cost rankings within each admission type show minimal variation. The CTE fix here is a strong interview talking point demonstrating PostgreSQL-specific knowledge.

Query 4.5 — Patient admission frequency tier (NTILE + CASE)

Question: Segment patients into frequency tiers based on how often they are admitted.

A CTE counts admissions per patient. NTILE(4) divides patients into 4 equal-sized buckets ordered by admission frequency DESC. A CASE statement converts the bucket number into a readable label. NTILE is written twice as PostgreSQL cannot reference a SELECT alias in the same SELECT.

Finding: The majority of patients fall into the Low Frequency tier with a single admission. High Frequency patients are the highest-cost cohort and the primary target for case management intervention in real insurance operations.

Section 5 — Combined Analysis (Queries 5.1–5.2)

Query 5.1 — Full Patient Risk Summary

Question: Produce an executive-level summary of each patient combining admissions, billing, stay length, and test outcomes.

A single CTE aggregates 7 metrics per patient — total admissions, lifetime billing, average billing, average stay days, abnormal result count, emergency admission count, and most recent admission date. Two CASE statements inside SUM() create conditional counts without requiring separate queries. RANK() and NTILE(5) are applied in the outer query to add billing rankings.

Finding: This is the most practically valuable query in the project — it produces the kind of patient-level risk profile a health insurer case manager would use daily. The billing_quintile column segments patients into five cost tiers, allowing the highest-cost 20% to be immediately identified for proactive outreach and care coordination.

Query 5.2 — Insurer Performance

Question: Rank each insurer across multiple dimensions — claims volume, revenue, avg billing, and proportion of high-cost claims.

A CTE computes 6 metrics per insurer including high_cost_claims (billing over \$40,000) using a conditional SUM with CASE. Three separate RANK() window functions in the outer query rank each insurer on revenue, volume, and average claim value independently — producing a multi-dimensional scorecard.

Finding: Insurers perform consistently across all three ranking dimensions. The pct_high_cost column — claims over \$40,000 as a percentage of total — is the most practically relevant metric, indicating which insurers carry the highest concentration of complex costly cases. This scorecard pattern is standard in insurance management reporting.